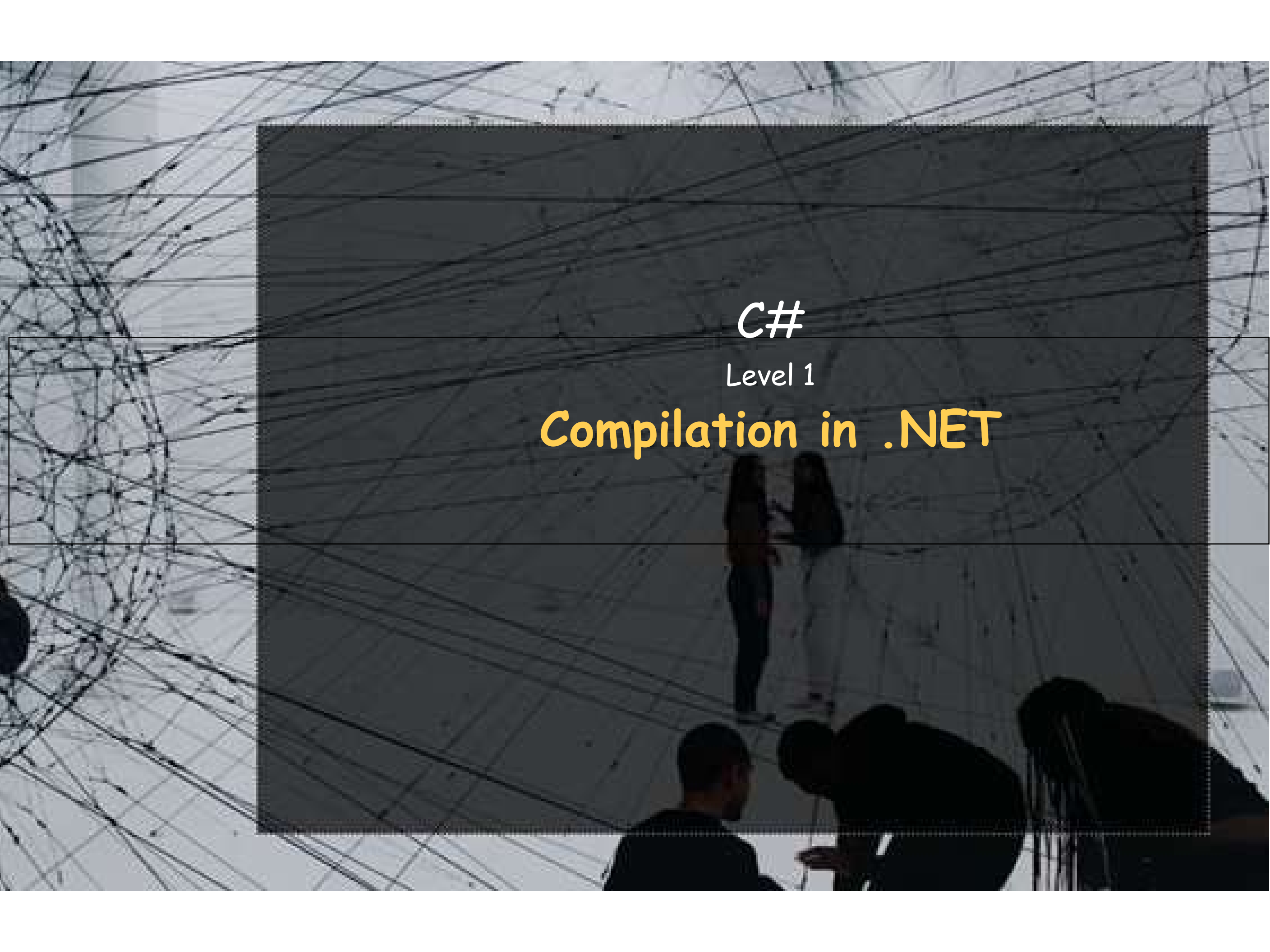


The background is a blurred image of a computer monitor. It shows a dark-themed code editor with various lines of code in different colors (blue, green, white). At the bottom, a Windows taskbar is visible with several application icons including Chrome, Photoshop, and others. The overall image is out of focus, serving as a backdrop for the text.

C#

Level 1

Compilation in .NET



C#

Level 1

Compilation in .NET

🔍 Definition

■ .NET Compilation

▨ Compilation in .NET is the process of converting source code written in languages such as C#, F#, or Visual Basic into code that can eventually be executed by the computer.

▨ The traditional execution model commonly involves compilation to Intermediate Language and then runtime compilation with JIT.

▨ Modern .NET also supports Ahead-of-Time compilation.

Explanation

When you write a *C#* program, the computer does not directly execute the *C#* source code.

C# / F# / VB



Language Compiler



IL / CIL



Assembly (.dll / .exe)



CLR



JIT Compiler



Machine Code



CPU

➡ Result: The source code eventually becomes native machine code that the processor can execute.

Explanation

CIL / IL

 CIL (Common Intermediate Language), also called IL, is an intermediate instruction set generated from .NET source code.

 It is CPU-independent.

 It is stored inside .NET assemblies together with metadata.

CLR

 CLR (Common Language Runtime) is the runtime environment responsible for executing managed .NET code.

 It provides services such as JIT compilation, garbage collection, exception handling, type safety, and memory management.

JIT

 JIT converts IL into native machine code at runtime.

Procedures

1. Write the source code.
2. Compile the source code using the appropriate language compiler.
3. Generate IL/CIL.
4. Store IL and metadata in an assembly such as .dll or .exe.
5. Load the assembly through the .NET runtime.
6. Use JIT when applicable to convert IL into native machine code.
7. Execute the generated machine code.

💡 Example

💡 Example

```
static void Hello()  
{  
    Console.WriteLine("Hello");  
}
```

C# Code



IL



Assembly



CLR / JIT



Native Machine Code



CPU

➡ Result:

The CPU executes the generated native code.

Note

- ◆ Modern .NET JIT behavior is more nuanced than simply compiling every method once.
- ◆ The runtime can use different compilation strategies and optimizations depending on the application and execution profile.
- ◆ AOT can generate native code before the application runs.

⊘ Warning

◆ Do not confuse IL with machine code.

▨ IL → Intermediate, CPU-independent instructions

▤ Machine Code → Native instructions for a specific CPU architecture

◆ Do not think that the C# compiler traditionally converts C# directly into machine code.

➡ The typical JIT-based flow is:

C#



IL



JIT



Machine Code



Comparison

▨ JIT — Just-In-Time

- ✓ Compiles IL at runtime
- ✓ Generates native code when needed
- ✓ Can optimize according to the runtime environment
- ✓ Requires runtime compilation work
- ✓ Common execution model for .NET

▤ AOT — Ahead-of-Time

- ✓ Compiles code before execution
- ✓ Native code is available earlier
- ✓ Can improve startup time
- ✓ Reduces runtime JIT work
- ✓ Useful for specific deployment scenarios

✦✦ Summary

- ✓ Source code is written in languages such as C#, F#, and Visual Basic
- ✓ The language compiler converts source code into IL/CIL
- ✓ IL is stored inside assemblies such as .dll and .exe
- ✓ CLR provides the runtime environment
- ✓ JIT converts IL into native machine code at runtime
- ✓ AOT generates native code ahead of execution
- ✓ JIT = Compile at runtime
- ✓ AOT = Compile before runtime

Thank you 😊